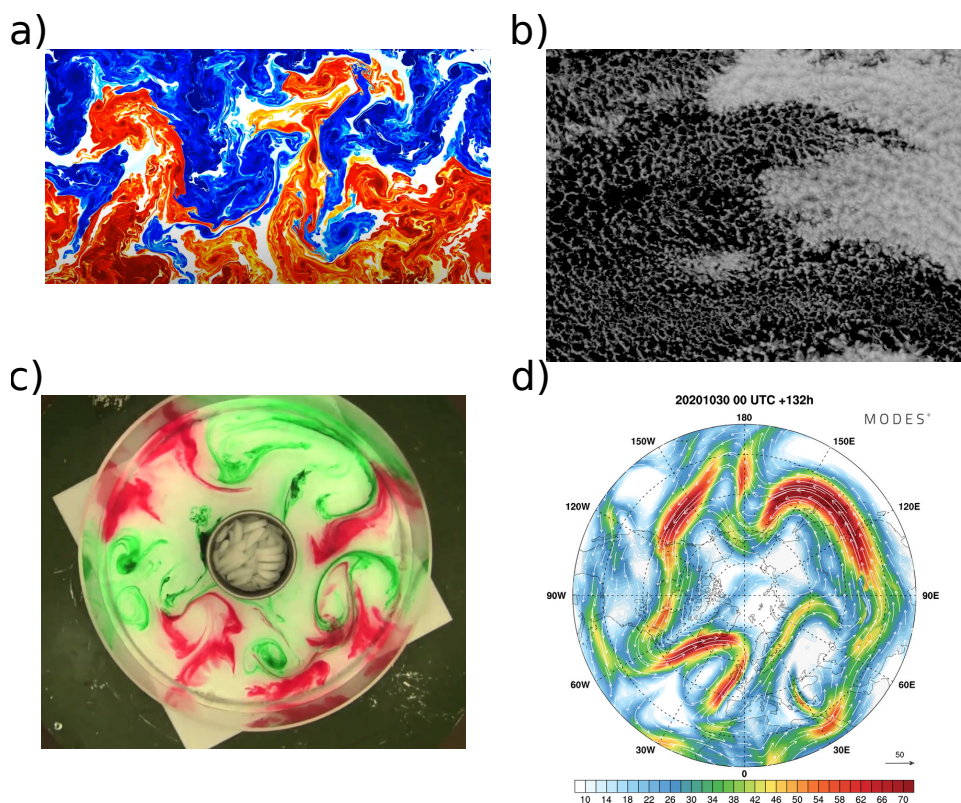


4. naloga: Integracija dinamičnih sistemov v času - Lorenzov '63 sistem

1 Uvod

Kaos je eno največjih odkritij teoretične fizike 20. stoletja, predvsem po zaslugi prebojnega članka ameriškega matematika in meteorologa Edwarda Lorenza iz leta 1963 [1]. Lorenz je študiral hidrodinamske sisteme, kot sta npr. konvekcija med dvema ploščama z različno temperaturo (Slika 1a) ali pa tok tekočine v rotirajočem cilindričnem bazenu, kjer sredico hladimo, plašč valja pa grejemo (Slika 1c). Slednji (t.i. Fultzov) eksperiment zelo dobro simulira planetarne Rossbyeve valove, s katerimi poteka prenos toplote v Zemljini atmosferi med subtropskimi in subpolarnimi predeli in ki ključno vplivajo na vreme v zmernih geografskih širinah (Slika 1d).



Slika 1: (a) Simulacija Rayleigh-Benardove konvekcije med dvema ploščama z različno temperaturo; (b) satelitski posnetek konvekcijskih celic v ozračju (temna so območja spuščanja zraka, svetla pa območja, kjer se zrak dviguje, zaradi ohlajanja ob dviganju pa pride do kondenzacije vodne pare in nastanka oblakov); (c) tok tekočine v cilindričnem bazenu, kjer sredico hladimo, plašč bazena pa grejemo; (d) vizualizacija Rossbyevih valov na 200 hPa pritiskovi ploskvi (na višini približno 11 km) nad severno poloblo [2]. Slike iz <https://www.youtube.com/watch?v=uWdKVpQ94Ns>, <https://baike.baidu.com/item/19115670>, <https://modes.cen.uni-hamburg.de/>.

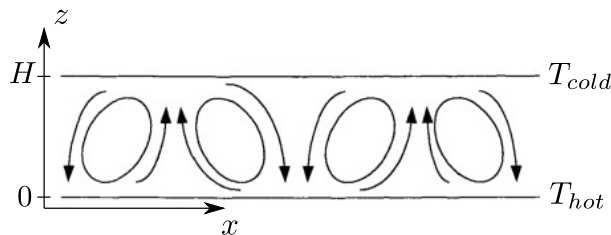
Lorenz je opazil, da so spremembe toka tekočine v nekaterih hidrodinamskih sistemih pri določenih pogojih neperiodične. To pomeni, da se neko stanje niti po daljšem času nikoli ne

ponovi eksaktno ali pa je ponovljivost rešitve omejena na kratko časovno obdobje. Ugotovil je, da so omejene in neperiodične rešitve običajno nestabilne že pri majhni spremembi začetnega pogoja. To pomeni, da dve trajektoriji rešitev, ki sta bili ob začetnem času blizu skupaj, sčasoma postaneta povsem različni. Ta efekt je slikovito opisal v naslovu enega izmed svojih predavanj: *Predictability: Does the flap of a butterfly's wings in Brazil set off a tornado in Texas?* Danes je učinek bolj poznan kot *učinek metulja*.

Dinamični sistem, ki ga je študiral Lorenz v [1], je poenostavljen opis konvekcije med dvema razsežnima planparalelnima ploščama z različnima temperaturama (Slika 2). Zaradi enostavnosti predpostavimo, da je tok tekočine med njima dvodimenzionalen: $\mathbf{v} = (u, v, w) = (u, 0, w)$. Opišemo ga z Navier-Stokesovo enačbo

$$\begin{aligned} \frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} &= -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{v} - g \hat{k} \\ &\cong -\frac{1}{\rho_0} \nabla p + \nu \nabla^2 \mathbf{v} + g \beta (T - T_0) \hat{k}, \end{aligned} \quad (1)$$

kjer je g gravitacijski pospešek, ρ gostota zraka, p je tlak v tekočini, $\nu = \mu/\rho$ pa je koeficient kinematične viskoznosti. Za gostoto zraka ρ po Boussinesqovem približku (ki velja pri majhnih premikih delca tekočine) predpostavimo, da je konstantna in enaka ρ_0 , variacije gostote pa upoštevamo le v členu za vzgon v vertikalni gibalni enačbi. V približku 1. reda predpostavimo, da je gostota linearno odvisna od temperature T . Za volumski raztezek snovi s temperaturo namreč velja $\beta = -\frac{1}{\rho} \left(\frac{\partial \rho}{\partial T} \right)_p \approx -\frac{1}{\rho_0} \frac{\rho - \rho_0}{T - T_0}$ in od tod $\rho = \rho_0 - \beta \rho_0 (T - T_0)$, kjer je ρ_0 referenčna gostota snovi pri temperaturi T_0 . Izpeljava zgornje enačbe je navedena v Dodatku A.



Slika 2: Skica konvektivnih celic med dvema ploščama z različnima konstantnima temperaturama. Glavni razlog za nastanek celic je vzgon, torej dviganje redkejšje tekočine, kjer se lokalno del tekočine pregreje napram okolici, in spuščanje hladnejše in gostejše tekočine navzdol. Enako deluje konvekcija v ozračju - ob jasnem dnevu se čez dan segrejejo tla zaradi sončevega sevanja, s prevajanjem pa se segreje tudi zrak povsem blizu tal. Ta se razteza, postane lokalno redkejši od okoliškega zraka in se prične dvigovati. Slika iz [3].

Prenos toplote v tekočini poteka s konvekcijo in difuzijo. Zapišemo

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \kappa \nabla^2 T, \quad (2)$$

kjer je κ konstanta toplotne difuzije, $\kappa = \lambda/(\rho c_p)$. V primeru, da bi tekočina mirovala ($\mathbf{v} = 0$), bi bil v stacionarnem stanju profil temperature med obema ploščama izključno posledica prevajanja toplote in torej linearen. Med zgornjo hladjeno ploščo pri temperaturi T_{cold} in spodnjo ogrevano ploščo T_{hot} , ki sta za H narazen, bi bil profil temperature $T_c(z) = T_{\text{hot}} - \Gamma z$, kjer je $\Gamma = (T_{\text{hot}} - T_{\text{cold}})/H$. Enačbo 2 nato prepišemo za temperaturne odmiike θ od linearne profila temperature, $\theta(x, z, t) = T(x, z, t) - T_c(z)$, in dobimo:

$$\frac{\partial \theta}{\partial t} - w \Gamma + \mathbf{v} \cdot \nabla \theta = \kappa \nabla^2 \theta. \quad (3)$$

S tem precej poenostavimo tudi robne pogoje za temperaturo (oz. zdaj za njen odmik):

$$\begin{aligned} \mathbf{v}(x, y, z) &= 0 \quad \text{za} \quad z = 0, H \\ \theta(x, y, z) &= 0 \quad \text{za} \quad z = 0, H \end{aligned} \quad (4)$$

Upoštevamo tudi, da je tok nestisljiv (brezdivergenten), torej velja $\nabla \cdot \mathbf{v} = 0$. Posledično lahko vektorsko polje hitrosti preprosto zapišemo s tokovno funkcijo, tako da velja $\mathbf{v} = \nabla \times \mathbf{\Psi}$, kjer je $\mathbf{\Psi} = (0, \psi(x, z), 0)^T$. PDE (1) za komponento u odvajamo po z , PDE za komponento w pa odvajamo po x , enačbi odštejemo, izrazimo u in w s tokovno funkcijo in dobimo:

$$\frac{\partial \nabla^2 \psi}{\partial t} - \frac{\partial \psi}{\partial z} \frac{\partial}{\partial x} (\nabla^2 \psi) + \frac{\partial \psi}{\partial x} \frac{\partial}{\partial z} (\nabla^2 \psi) = \nu \nabla^2 (\nabla^2 \psi) + \beta g \frac{\partial \psi}{\partial x}. \quad (5)$$

Enačbo 3 pa s tokovno funkcijo zapišemo kot:

$$\frac{\partial \theta}{\partial t} - \frac{\partial \psi}{\partial z} \frac{\partial \theta}{\partial x} + \frac{\partial \psi}{\partial x} \frac{\partial \theta}{\partial z} = \kappa \nabla^2 \theta + \Gamma \frac{\partial \psi}{\partial x}. \quad (6)$$

Rešitev poiščemo z 2D Fourierovim razvojem

$$\begin{aligned} \psi(x, z, t) &= \sum_{m,n} \psi_{m,n}(t) \exp(im\pi x/L) \sin(n\pi z/H) \\ \theta(x, z, t) &= \sum_{m,n} \theta_{m,n}(t) \exp(im\pi x/L) \sin(n\pi z/H), \end{aligned} \quad (7)$$

kjer je L perioda oscilacij v smeri x . S sinusnim nastavkom v vertikalni smeri pa že zadostimo robnim pogojema (4). Tako dobimo neskončen sistem sklopljenih navadnih diferencialnih enačb za koeficiente $\psi_{m,n}$ in $\theta_{m,n}$. Če želimo dobiti minimalen še vedno nelinearen sistem enačb, moramo vzeti najmanj koeficiente $\alpha X(t) = \text{Im}(\psi_{11}(t))$ in $\beta Y(t) = \text{Re}(\theta_{11}(t))$ ter $\gamma Z(t) = \text{Re}(\theta_{02}(t))$:

$$\begin{aligned} \psi(x, z, t) &= \alpha X(t) \sin(\pi x/L) \sin(\pi z/H) \\ \theta(x, z, t) &= \beta Y(t) \cos(\pi x/L) \sin(\pi z/H) + \gamma Z(t) \sin(2\pi z/H). \end{aligned} \quad (8)$$

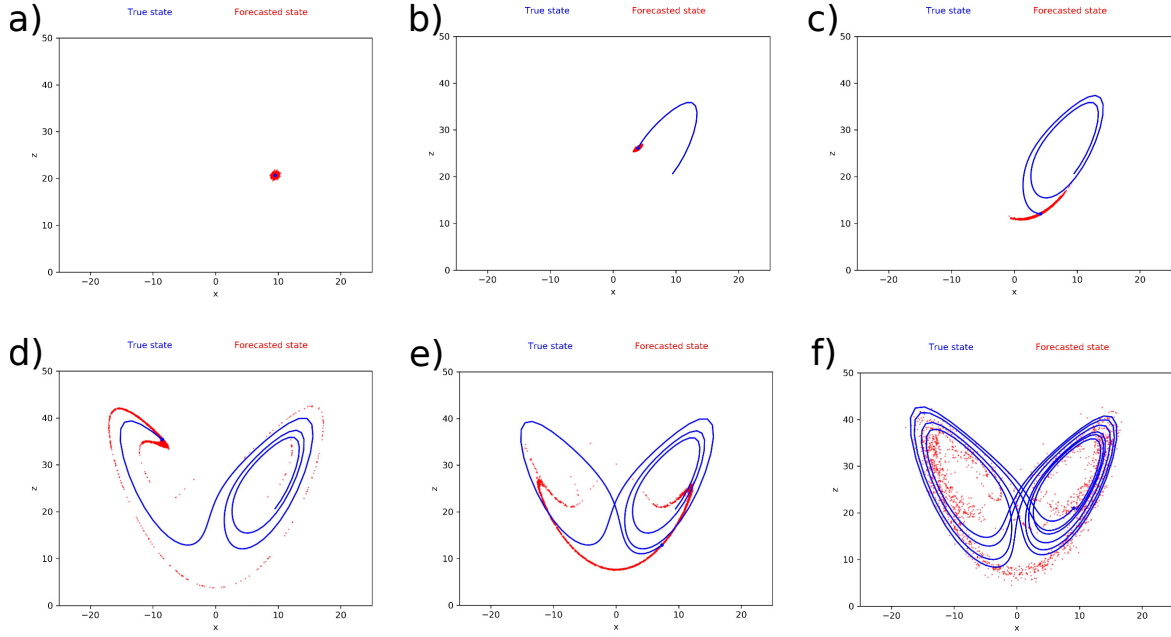
Z upoštevanjem Rayleighovega števila $R = g\beta H^3(T_{\text{hot}} - T_{\text{cold}})/(\nu\kappa)$, njegove kritične vrednosti $R_c = \pi^4(1 + a^2)^3/a^2$, kjer je $a = H/L$, in reskaliranjem časa $\tau = \pi^2 \frac{1+a^2}{H^2} \kappa t$ končno dobimo Lorenzov '63 reduciran sistem enačb v brezdimenzijskih količinah.

$$\begin{aligned} \frac{dX}{d\tau} &= \sigma(Y - X) \\ \frac{dY}{d\tau} &= X(r - Z) - Y \\ \frac{dZ}{d\tau} &= XY - bZ, \end{aligned} \quad (9)$$

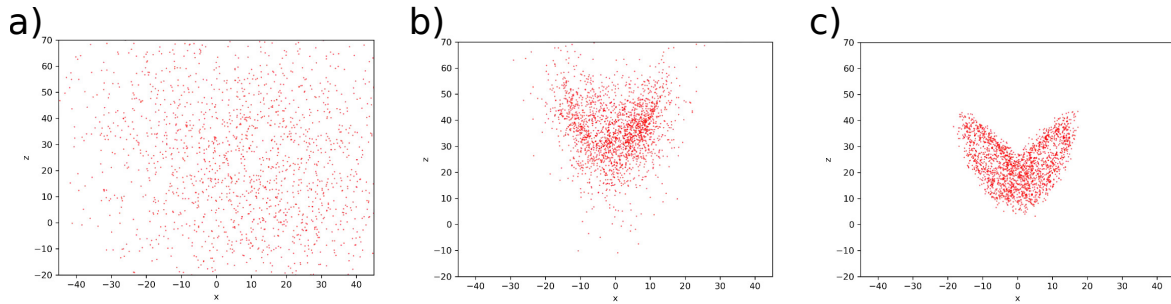
Parameter $\sigma = \nu/\kappa$ je Prandtlovo število, $r = R/R_c$ je reskalirano Rayleighovo število, $b = 4/(1 + a^2)$ pa je odvisen od vertikalne razsežnosti plasti. Vsi parametri so pozitivni. Kadar je $r < 1$ ($R < R_c$), ni konvekcijskega toka in prenos toplote poteka izključno s prevajanjem, kadar pa $r > 1$, pride do prenosa toplote s konvekcijo.

Pri vrednostih parametrov, kot jih je uporabil Lorenz, t.j. $\sigma = 10$, $b = 8/3$ in $r = 28$, je dinamični sistem:

- nelinearen (vsebuje produkte prognostičnih spremenljivk, npr. XZ in XY),
- neperiodičen (orbite, trajektorije v faznem prostoru se ne ponavljajo in se ne stekajo proti fiksni točki, ko $t \rightarrow \infty$),



Slika 3: Razvoj resničnega stanja in ansambel $N = 2000$ modelskih napovedi z rahlo perturbiranimi začetnimi pogoji. Ti se nahajajo znotraj neke hipersfere (a). Hipervolumen se sčasoma deformira v elipsoid (b). V nestabilnem sistemu se bo vsaj ena od osi elipsoida s časom povečevala in deformirala zaradi nelinearnih efektov (c). Nato pride do raztrganja elipsoida (d) - v tej fazi je verjetnostna napoved še vedno lahko uspešna. V fazi (e) izgubimo napovedljivost, v fazi (f) pa zgolj vemo, da je rešitev nekje na atraktorju, torej znotraj “klimatologije” rešitev.



Slika 4: (a) Fazni prostor točk v Lorenzovem sistemu ob začetnem času $t = t_0$, (b) v prehodni fazi ob $t_1 > t_0$ in (c) po dolgem času $t \gg 0$.

- determinističen (nima naključnih ali zašumljenih parametrov),
- izjemno občutljiv na spremembe v začetnem pogoju (bližnje trajektorije divergirajo eksponentno hitro, Slika 3). Dolgoročno napovedovanje stanja je ob neeksaktnih začetnih pogojih nemogoče. Enak problem imamo pri napovedovanju vremena. Hitrost divergence dveh trajektorij, ki sta ob začetnem času infinitezimalno blizu skupaj (njuna razlika je $\delta \mathbf{x}(0)$), opišemo z Ljapunovim eksponentom λ , tako da v začetnem linearnem režimu velja

$$|\delta \mathbf{x}(t)| \approx e^{\lambda t} |\delta \mathbf{x}(0)|. \quad (10)$$

Število Ljapunovih eksponentov je enako dimenziji faznega prostora (v našem primeru 3). V primeru večdimenzionalnih sistemov spremljamo Ljapunov spekter. Če je vsaj

en Ljapunov eksponent $\lambda_i > 0$, pomeni, da je sistem kaotičen in da razlika med dvema trajektorijama narašča eksponentno, torej bosta sčasoma postali povsem različni. V tem primeru se bo hiperelipsoid na Sliki 3b vsaj v eni smeri še naprej raztezal. Največji in hkrati nestabilen eksponent

$$\lambda_{max} = \lim_{t \rightarrow \infty} \lim_{|\delta \mathbf{x}(0)| \rightarrow 0} \frac{1}{t} \ln \frac{|\delta \mathbf{x}(t)|}{|\delta \mathbf{x}(0)|} \quad (11)$$

najhitreje narašča in po dolgem času povsem dominira Ljapunov spekter. Največji eksponent določa tipičen časovni okvir napovedljivosti, torej $\tau = 1/\lambda_{max}$.

- disipativen (nasprotno od Hamiltonskih sistemov, ki skupno energijo ohranjajo). To pomeni, da se bo trajektorija, ki se začne v poljubni začetni točki, po prehodnem obdobju sčasoma ustalila na atraktorju (Slika 4). Atraktor Zemljine atmosfere je na primer kar njena klima. Disipativnost pokažemo tako, da najprej posamezno trajektorijo opišemo s $\frac{\partial \mathbf{X}}{\partial t} = \mathbf{F}(\mathbf{X}, t)$, nato pa izračunamo divergenco trajektorij množice začetnih točk, ki določajo nek hipervolumen V . Velja $\frac{dV}{dt} = \int_V \nabla \cdot \mathbf{F} dV$, kjer je

$$\nabla \cdot \mathbf{F} = \frac{\partial \dot{X}}{\partial X} + \frac{\partial \dot{Y}}{\partial Y} + \frac{\partial \dot{Z}}{\partial Z} = -(\sigma + b + 1) < 0, \quad (12)$$

od tod pa $V(t) = V_0 \exp(-(\sigma + b + 1)t)$. Trajektorije torej konvergirajo in začetni volumen se sčasoma zmanjša na globalno atraktorsko množico z volumnom 0.

Zakaj bi sploh želeli Lorenzov sistem reševati z nevronske mreže? Lorenzov '63 model je preprost dinamični sistem, ki pa je, podobno kot atmosfera, nelinearen in kaotičen. V meteorologiji zato Lorenzov '63 model ali pa njegove zapletenejšje različice, ki simulirajo procese na različnih velikostnih skalah (Lorenz '96), uporabljamo za testiranje asimilacijskih algoritmov in numeričnih metod. Z njim se lahko tudi precej naučimo o morebitnih težavah, ki nas čakajo pri kasnejšem *data-driven* napovedovanju vremena. Glavni razlog za vpeljavo strojnega učenja v numerično napovedovanje vremena je želja po pohitritvi in še večji paralelizaciji visokoločljivostnih izračunov, s čimer bi si lahko privoščili večje ansamble za verjetnostno napovedovanje vremena. V veliko aspektih so metode strojnega učenja tudi uspešnejše od klasičnih pristopov.

2 Metoda

2.1 Rekurentna nevronska mreža - RNN

Rekurentne nevronske mreže se uporabljajo za procesiranje multivariatnih časovnih vrst $\mathbf{x}(0), \mathbf{x}(1), \dots, \mathbf{x}(n)$, kjer v Lorenzovem '63 sistemu $\mathbf{x}(t)$ predstavlja 3-dimenzionalni vektor stanja ob časovnem koraku t . Še posebej so uporabne za napoved pojavov, katerih prihodnje stanje je strogo determinirano s prejšnjim stanjem, kar velja tudi za naš dinamični sistem.

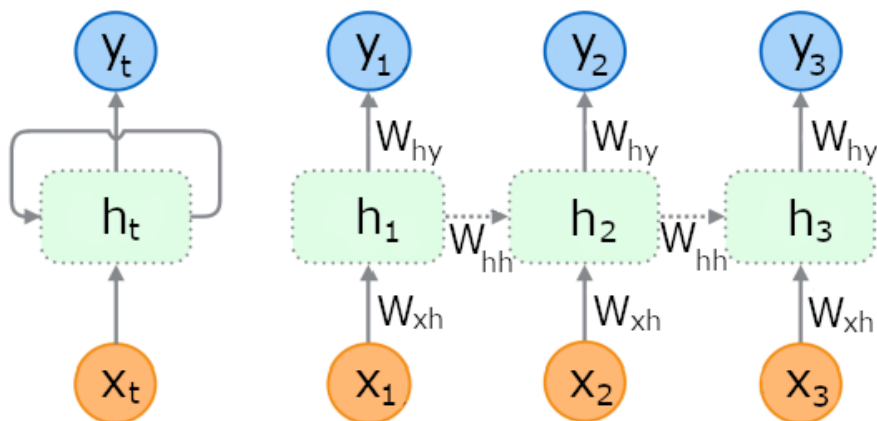
Rekurentna nevronska mreža sestoji iz vektorja skritega stanja $\mathbf{h}_t$, na katerega z ločenima gosto povezanima nevronska mrežama preslikamo vhodni vektor $\mathbf{x}_t$ in prejšnje skrito stanje $\mathbf{h}_{t-1}$ (ničto skrito stanje lahko inicializiramo kot vektor ničel). Skrito stanje se rekurentno posodablja po vsakem novem prebranem vhodnem vektorju. Zapišemo torej ponavljajoč (rekurenten) vzorec

$$\begin{aligned} \mathbf{h}_t &= f(\mathbf{h}_{t-1}, \mathbf{x}_t; \mathbf{W}) \\ \mathbf{h}_{t+1} &= f(\mathbf{h}_t, \mathbf{x}_{t+1}; \mathbf{W}) \\ &\vdots \end{aligned} \quad (13)$$

kjer f predstavlja linearni preslikavi z matrikama uteži in aktivacijsko funkcijo. Skrito stanje torej deluje kot spomin nevronske mreže, saj vsako novo stanje vsebuje informacijo o prekelih vhodnih podatkih. V izhodni plasti vektor skritega stanja $\mathbf{h}_t$ z gosto povezano nevronske mrežo preslikamo v izhodni vektor $\mathbf{y}_t$, ki ima ustrezne dimenzije (Slika 5). Najpreprostejši matrični zapis procesa je sledeč

$$\begin{aligned}\mathbf{h}_t &= \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t) \\ \mathbf{y}_t &= \mathbf{W}_{hy}\mathbf{h}_t.\end{aligned}\tag{14}$$

Aktivacija $\tanh$ deluje na vsako komponento vektorja posamično in regulira, da vrednosti, ki tečejo skozi mrežo, ne presežejo intervala $(-1, 1)$. Uteži $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$ in $\mathbf{W}_{hy}$ so enake za vse časovne korake, kar zmanjša število parametrov modela in verjetnost preprežganja učnim podatkom.



Slika 5: Shema ene zanke RNN (levo) in časovno razpotegnjen prikaz napovedi ene plasti RNN, kjer množica vhodnih podatkov določa množico izhodov (*many-to-many prediction*). Če pa npr. zaporedje vhodov določa le zadnji izhod, potem imamo napoved *many-to-one*. V tej nalogi bomo uporabljali le pristop *many-to-one*. Slika prirejena po [4].

Rekurentno posodabljanje skritih stanj povzroča hitro izgubo informacij o preteklih vhodih, zato pravimo, da imajo kratkoročen spomin. Pri daljših časovnih zaporedjih torej RNN težko prenesejo informacijo iz začetnih časovnih korakov na končne časovne korake. Posledično je informacija iz časa dlje nazaj komaj upoštevana. Problem se pojavi tudi med učenjem, kjer med propagacijo gradientov v času nazaj pride do velikega števila množenj, zaradi česar gradienti konvergirajo bodisi k majhnim bodisi velikim vrednostim. Posledično se uspešno posodabljanje uteži in s tem tudi učenje nevronske mreže konča.

2.1.1 LSTM

Problem kratkoročnega spomina in izginjajočih oz. eksplodirajočih gradientov rešujejo mreže tipa *Long short-term memory* (LSTM). Te poleg skritega stanja $\mathbf{h}_t$ vsebujejo tudi vektor celičnega stanja $\mathbf{c}_t$, ki skrbi za dolgoročni spomin. Tok informacij med stanjema je kontroliran z vhodnimi (*input*), izhodnimi (*output*) in spominskimi (*forget*) vrati. Vrata so gosto povezane nevronske mreže, katerih naloga je kontrolirati količino starih informacij v celičnem stanju (spominska vrata), dodajati nove informacije celičnemu stanju (vhodna vrata) ali generiranje izhoda skritega stanja (izhodna vrata). Sigmoidna aktivacijska funkcija s preslikavo vrednosti med 0 in 1 določa relevantnost posameznih komponent. Takšna arhitektura upočasni izgubljanje dolgoročnih informacij, hkrati pa zmanjša težavo izginjajočih oz. eksplodirajočih gradientov med učenjem.

2.1.2 Luongov mehanizem pozornosti

Težavo kratkoročnega spomina lahko rešimo tudi z uporabo Luongovega mehanizma pozornosti [5]. Ta se osredotoča na skrita stanja, zato deluje za vse tipe rekurentnih nevronske mreže, ki pa morajo biti sestavljene iz kodirnika (*encoder*) in dekodirnika (*decoder*). Kodirnik prejme časovno vrsto vhodnih vektorjev $\mathbf{x}_i$ in jih kodira v skrita stanja $\mathbf{h}_i$, kjer velja $i \in \{1, 2, \dots, T\}$. Zadnje skrito stanje kodirnika in prvi vhodni vektor dekodirnika (t. i. *start token* - npr. vektor ničel ali zadnji vhodni vektor kodirnika) sta uporabljena za inicializacijo dekodirnika, ki izračuna dekodirna skrita stanja. Običajne RNN bi ta skrita stanja $\mathbf{h}_t$ z gosto povezano nevronske mreže preslikale v napoved $\mathbf{y}_t$. Luongov mehanizem pozornosti namesto tega primerja tarčno skrito stanje $\mathbf{h}_t$ z vsemi kodirnimi skritimi stanji $\mathbf{h}_i$ in konstruira nov vektor, t. i. vektor konteksta, ki je s pozornostjo utežena linearna kombinacija kodirnih skritih stanj. Tega nato združimo s tarčnim skritim stanjem in preslikamo v novo - s kontekstom obogateno - skrito stanje $\mathbf{H}_t$, katerega spomin o oddaljenih skritih stanjih je osvežen.

Za izračun pozornosti najprej definiramo način primerjave skritih stanj, t. i. *score*. V ta namen lahko uporabimo skalarni produkt

$$\text{score}(t, i) = \mathbf{h}_i \cdot \mathbf{h}_t, \quad (15)$$

s katerim merimo podobnost (mero ortogonalnosti) med stanjema. Tega izračunamo za vsako skrito stanje kodirnika i , s čimer dobimo podobnost med tarčnim in vsemi kodirnimi skritimi stanji. Te vrednosti nato pretvorimo v uteži pozornosti. Za probabilistično interpretacijo želimo, da imajo uteži pozornosti vrednosti med 0 in 1 in da se seštejejo v 1. V ta namen lahko pozornost med $\mathbf{h}_t$ in $\mathbf{h}_i$ izračunamo kot

$$a(t, i) = \frac{\exp(\text{score}(t, i))}{\sum_{j=1}^T \exp(\text{score}(t, j))}. \quad (16)$$

Če zapišemo $\mathbf{scores} = (\text{score}(t, 1), \dots, \text{score}(t, T))$, prepoznamo, da lahko vse vrednosti pozornosti na podlagi enačbe 16 zapišemo kot $\mathbf{a} = \text{softmax}(\mathbf{scores})$, kjer je $\mathbf{a} = (a(t, 1), \dots, a(t, T))$ vektor vseh uteži pozornosti. Vektor konteksta za tarčno skrito stanje $\mathbf{h}_t$ tedaj izračunamo kot

$$\mathbf{c}_t = \sum_{i=1}^T a(t, i) \mathbf{h}_i. \quad (17)$$

V tem primeru dolžina vhodnega časovnega zaporedja iz stališča modelskega spomina ni več težava, saj lahko tarčna vrednost posveča pozornost tudi začetnemu delu časovne vrste. Preostane nam še izračun novega skritega stanja:

$$\mathbf{H}_t = \tanh \left(W_c \begin{bmatrix} \mathbf{c}_t \\ \mathbf{h}_t \end{bmatrix} \right), \quad (18)$$

kjer je $\begin{bmatrix} \mathbf{c}_t \\ \mathbf{h}_t \end{bmatrix}$ vektor konkatencije (zlepek) vektorja konteksta in tarčnega skritega stanja.

Mehanizem pozornosti je uporabljen tudi v transformerjih, kjer ga namesto na vektorjih skritih stanj uporabimo neposredno na vhodnih vektorjih, od koder tudi ime originalnega članka o transformerjih "Attention is all you need" [6].

3 Implementacija

Najprej pripravimo učne podatke, kot je razloženo v dodatku B. Podatke normaliziramo tako, da jim odštejemo povprečno vrednost in jih delimo s standardno deviacijo (t. i. standardizacija). Ker so zaloge vrednosti vseh treh komponent precej različne, za vsako komponento uporabimo svoje normalizacijske vrednosti. Uporabili bomo knjižnico scikit-learn:

```

import numpy as np
from sklearn.preprocessing import StandardScaler

# Uporabimo standardizacijo (opcija je tudi MinMaxScaler)
scaler = StandardScaler()
# Povprečje in std izračunamo na učni množici (ne na validacijski)!
scaler.fit(X_train)

# Izpiši povprečno vrednost in standardno deviacijo
print("Povprečje:", scaler.mean_)
print("Standardna deviacija:", scaler.scale_)

# Normiramo podatke
X_train_norm = scaler.transform(X_train)
X_val_norm = scaler.transform(X_val)

```

3.1 Sekvenciranje podatkov

Rekurentne nevronske mreže kot vhod uporabijo sekvenco oz. zaporedje vhodnih vektorjev. Izhod RNN je lahko bodisi en sam vektor (*many-to-one*) bodisi zaporedje vektorjev (*many-to-many*). V nalogi uporabljamo le pristop *many-to-one*, kjer v model iterativno vstavljamo *input_seq_len* preteklih časovnih stanj (vključno s preteklimi napovedmi) in vsakič generiramo skrita (in celična) stanja ter preslikamo v napoved:

$$\begin{bmatrix} \mathbf{X}(t) \\ \mathbf{X}(t - \Delta t) \\ \mathbf{X}(t - 2\Delta t) \\ \vdots \\ \mathbf{X}(t - (k - 1)\Delta t) \end{bmatrix} \xrightarrow{\mathcal{R}_{NN}} \mathbf{X}(t + \Delta t); \quad \mathbf{X}(t) = (X(t), Y(t), Z(t)). \quad (19)$$

Učno množico pripravimo tako, da so vhodi v RNN sekvence dolžine *input_seq_len*, izhodi pa dolžine 1.

```

# Sekvenciranje multivariatnega zaporedja
def split_into_sequences(
    multivariate_sequence, input_seq_len,
    start_every_num_steps=8):
    """
    multivariate_sequence: shape (T, d); T=# čas. korakov, d=# dimenzij
    input_seq_len: dolžina vhodnega zaporedja
    start_every_num_steps: koliko časovnih korakov izpustimo,
                          preden shranimo novo sekvenco
    """
    X, y = list(), list()
    # Zanka skozi vse časovne korake
    for i in range(0, len(multivariate_sequence), start_every_num_steps):
        # Zadnji indeks sekvence
        end_ix = i + input_seq_len
        # Preveri če je zadnji indeks izven multivariate_sequence
        if end_ix > len(multivariate_sequence)-1:

```



```

        break
    # seq_x je sestavljena iz input_seq_len korakov
    seq_x = multivariate_sequence[i:end_ix, :]
    # seq_y je samo 1 element za napoved
    seq_y = multivariate_sequence[end_ix, :]
    X.append(seq_x)
    y.append(seq_y)

return np.array(X), np.array(y)

input_seq_len = 5
# Učni podatki
X1_train, Y1_train = split_into_sequences(X_train_norm, input_seq_len)
# Validacijski podatki
X1_val, Y1_val = split_into_sequences(X_val_norm, input_seq_len)

```

Učne podatke je treba tudi naključno premešati, ampak bomo poskrbeli, da bo to tensorflow storil namesto nas. Če bi se kdo odločil preizkusiti tudi gosto povezane nevronske mreže, velja omeniti, da tedaj podatkov ne sekvencirate, ampak samo zamaknete v obliki `x_train = X_train_norm[0:-1, :]` in `y_train = X_train_norm[1:, :]` (analogno tudi za validacijsko množico).

3.2 Učenje nevronske mreže

Najpreprosteje nevronske mrežo definiramo kot zaporedje plasti z uporabo t. i. *Sequential API*. Če želimo imeti več zaporednih rekurentnih plasti, nastavimo `return_sequences=True`, kar omogoči, da naslednja plast namesto samo končnega prejme tudi vsa vmesna skrita stanja:

```

import tensorflow as tf
from tensorflow.keras.layers import Dense, SimpleRNN, LSTM
model = tf.keras.models.Sequential()
model.add(SimpleRNN(
    units=16, # dimenzija skritega stanja
    return_sequences=False,
    input_shape=(None,3))
)
# ekvivalent za LSTM: model.add(LSTM(...))
# Zadnje skrito stanje še preslikamo v izhodni vektor y
model.add(Dense(3))
# Nastavitve za učenje modela
model.compile(loss='mean_squared_error', optimizer='Adam')

```

Zgoraj definirana rekurentna mreža ima eno rekurentno plast s 16 dimenzionalnim skritim stanjem, katere izhodno skrito stanje gre v gosto povezano plast, ki ga preslika v 3 dimezionalen izhod (glej enačbo 14). Z nastavitvijo `input_shape=(None,3)` smo zagotovili, da bo naša nevronska mreža delovala ne glede na dolžino vhoda `input_seq_len` (pomembno je le, da je vhodni tenzor oblike `(batch_size, input_seq_len, features)` in da ima zadnja dimenzija velikost 3 (`features=[X, Y, Z]`)).

V nalogi bomo sestavili tudi LSTM mrežo z Luongovim mehanizmom pozornosti. V tem primeru bomo namesto Sequential API uporabili bolj fleksibilen Functional API, kjer definiramo vhodni tenzor, zapišemo arhitekturo nevronske mreže kot funkcijo, ki preslika vhodni tenzor

v izhodnega, in vse 3 zapakiramo v tensorflow model. Podrobnosti o teoriji tega modela so zapisane v poglavju 2.1.2. Najprej definiramo kodirnik.

```
import tensorflow as tf
from tensorflow.keras import layers

# KODIRNIK
# Vhodni tenzor za kodirnik
encoder_inputs = tf.keras.Input(shape=(None, 3))
# Kodirni sloj
encoder_lstm = layers.LSTM(
    units=16,                # Število dimenzij skritega stanja
    return_sequences=True,   # Vrni vsa skrita stanja
    return_state=True,       # Vrni zadnje celično stanje
    name='encoder')          # Sloj poimenujemo za kasnejšo rabo

# Pošlji vhodni tenzor kodirnika skozi kodirnik
encoder_hidden_states, state_h, state_c = encoder_lstm(encoder_inputs)
# encoder_hidden_states shape: # (batch_size, encoder_time_steps, hidden_size)
```

Z njim dobimo skrita stanja kodirnika za izračun pozornosti in zadnje skrito ter celično stanje za inicializacijo stanja dekodirnika. Nato definiramo dekodirnik.

```
# DEKODIRNIK
# Vhodni tenzor za dekodirnik
decoder_inputs = tf.keras.Input(shape=(None, 3))
# Dekodirni sloj
decoder_lstm = layers.LSTM(
    units=16,                # Število dimenzij skritega stanja
    return_sequences=True,   # Vrni vsa skrita stanja
    return_state=True,       # Vrni zadnje celično stanje
    name='decoder')          # Sloj poimenujemo za kasnejšo rabo

# Nastavi začetno skrito in celično stanje dekodirnika
# in pošlji vhodni tenzor dekodirnika skozi dekodirnik
decoder_hidden_states, _, _ = decoder_lstm(
    decoder_inputs, initial_state=[state_h, state_c])
# decoder_hidden_states shape: # (batch_size, decoder_time_steps, hidden_size)
```

Na tem mestu imamo vse, kar potrebujemo za izračun pozornosti, tj. skrita stanja kodirnika in dekodirnika. Sedaj kličemo funkcijo za izračun pozornosti in vektorja konteksta.

```
# Luongov mehanizem pozornosti (Luong Attention)
attention_layer = LuongAttention()
context_vector, attention_weights = attention_layer(
    decoder_hidden_states, encoder_hidden_states)
```

V ta namen pred modelom (nad definicijo kodirnika in dekodirnika) ustvarimo razred, v katerem definiramo sloje za izračun pozornosti:

```
# Custom layer for Luong Attention
class LuongAttention(tf.keras.layers.Layer):
```

```

def __init__(self):
    super(LuongAttention, self).__init__()

def call(self, decoder_hidden, encoder_outputs):

    # Izračun skalarnega produkta med skritim stanjem dekodirnika
    # in skritimi stanji kodirnika. V ta namen za vsak primer v batchu
    # množimo matriko z vrsticami skritih stanj kodirnika s
    # transponirano matriko skritih stanj dekodirnika (tf.matmul).
    score = tf.matmul(encoder_outputs, decoder_hidden, transpose_b=True)
    # shape = (batch_size, time_steps_encoder, time_steps_decoder=1)

    # Izračunamo uteži pozornosti - softmax po osi kodirnih skritih stanj
    attention_weights = tf.nn.softmax(score, axis=1)
    # shape = (batch_size, time_steps_encoder, time_steps_decoder=1)

    # Vektor konteksta je s pozornostjo utežena linearna kombinacija
    # izhodov kodirnika.
    context_vector = tf.reduce_sum(
        attention_weights * encoder_outputs, axis=1)
    # shape = (batch_size, hidden_size)

    return context_vector, attention_weights

```

Po izračunu vektorja konteksta, tega konkatenujemo s skritimi stanji kodirnika, nov vektor preslikamo v posodobljeno skrito stanje dekodirnika in ga preslikamo v izhod nevronske mreže. Nato ustvarimo model in konfiguriramo nastavitve za učenje:

```

# Konkatenujemo vektorje konteksta s skritimi stanji dekodirja
combined_context = layers.Concatenate(axis=-1)(
    [decoder_hidden_states, tf.expand_dims(context_vector, 1)])

# Konkatenuirane vektorje preslikamo v nova skrita stanja, tako da
# jih pošljemo skozi gosto povezano NN. V ta namen uporabimo sloj
# TimeDistributed, ki aplikira uteži na vse izhode.
new_decoder_hidden_states = layers.TimeDistributed(
    layers.Dense(units=16, activation="tanh"))(combined_context)

# Nova skrita stanja z gosto povezano NN preslikamo v izhode.
outputs = layers.TimeDistributed(
    layers.Dense(3))(new_decoder_hidden_states)

# Ustvarimo model (funkcijo oz. računski graf), tako da definiramo
# vhodna in izhodna polja.
model = tf.keras.Model(
    inputs=[encoder_inputs, decoder_inputs],
    outputs = outputs)

# Konfiguriramo nastavitve za učenje
model.compile(loss='mean_squared_error', optimizer='Adam')

```

V tej implementaciji smo model opremili tudi za napoved *many-to-many*. Če želimo posegati v učenje modela lahko definiramo t. i. `callback`, s katerim lahko nastavimo načrtovanje stopnje učenja ali predčasno zaustavitev učenja ob konvergenci.

```
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=8, verbose=0, mode='min')

reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss', factor=0.4, patience=8, min_lr=1e-4)
```

Model učimo z ukazom `.fit` in shranimo naučene uteži:

```
fit = model.fit(
    x=X1_train,
    y=Y1_train,
    batch_size=300,
    epochs=30,
    validation_data=(X1_val, Y1_val),
    callbacks=[reduce_lr, early_stop],
    shuffle=True)

# Učenje Luong modela:
# -> inputi in outputi so v enaki obliki, kot v tf.keras.Model:
# -> x=[X1_train, X1_train[:, -1:]]
# -> y=np.expand_dims(Y1_train, axis=1)
# -> validation_data=(X1_val, X1_val[:, -1:]), np.expand_dims(Y1_val, axis=1))

model.save('pot_do_modela')
```

V učenju smo nastavili `shuffle=True`, da se učni podatki premešajo, s čimer preprečimo preprileganje na podmnožico učnih podatkov. Do vrednosti funkcije izgube na učni in validacijski množici pri različnih epohah dostopamo z ukazoma `fit.history['loss']` in `fit.history['val_loss']`. Sposobnost posploševanja naučenega modela testiramo na novih, modelu doslej nevidenih podatkih. V ta namen generiramo N začetnih pogojev za testiranje.

```
# Število začetnih pogojev
N = 200

# Seme za ponovljivost eksperimenta
np.random.seed(10043)

# Začetni pogoji za učno trajektorijo
X0_test = float(np.random.uniform(low=-50., high = 50., size = 1))
Y0_test = float(np.random.uniform(low=-50., high = 50., size = 1))
Z0_test = float(np.random.uniform(low=-50., high = 50., size = 1))

# Čas integracije
t_test = 30.0 + N

# Numerična integracija
t_test, X_test = integratorLorenz63(X0_test, Y0_test, Z0_test, t_test)
```

```

# Relaksacija: odrežemo prvih 30.0 enot časa
relaksacijski_indeks = 3001 # 1, ker je vključen tudi začetni pogoj
X_test = X_test[relaksacijski_indeks:, :]

# Pripravimo začetne pogoje (ics=initial conditions)
X_ics = X_test[:N*100].reshape(N, 100, 3)
# Normiramo začetne pogoje
X_ics_norm = scaler.transform(X_test)[:N*100].reshape(N, 100, 3)
# X_ics(_norm).shape: (N, 100, 3)

```

Dobili smo N sekvenc dolžine 100. Vhod v nevronska mreža je zadnjih `input_seq_len` korakov sekvenc `X_ics_norm`. Za napoved iz n -tega začetnega pogoja z numeričnim integratorjem uporabimo začetne pogoje $X_0 = X_ics[n, -1, 0]$, $Y_0 = X_ics[n, -1, 1]$, $X_0 = X_ics[n, -1, 2]$ in $t_predict = 10$. Napovedovanje z nevronska mreža lahko izvedemo na sledeči način:

```

def napoved(N, model, input_seq_len, X_ics_norm, Luong_attention=False):
    """
    Funkcija za izračun modelskih napovedi
    args:
        - N: število začetnih pogojev
        - model: rekurentni model
            model = tf.keras.models.load_model(path_model)
        - input_seq_len: dolžina vhodne sekvence v model
        - Luong_attention (bool): True, če imamo model z Luongovo pozornostjo
    """
    # Priprava vhodnega tenzorja
    X_start = np.expand_dims(X_ics_norm[:, -input_seq_len:], axis=1)
    for j in range(input_seq_len-1, 0, -1):
        X_start = np.append(
            X_start, np.expand_dims(X_ics_norm[:, -j], axis=1),
            axis=1)

    # Tabela za zapisovanje napovedi - prva vrednost je začetni pogoj
    pred = X_start[:, -1:]

    if Luong_attention:
        start_token = X_start[:, -1:]

    # Izvedba napovedi za 10 enot časa
    T = [0.]
    for t in range(10*100):
        if Luong_attention == False:
            # Brez Luongove pozornosti:
            next_step = model.predict(X_start)
            next_step = np.expand_dims(next_step, axis=1)
            pred = np.append(pred, next_step, axis=1)
            X_start = np.concatenate((X_start[:, 1:], next_step), axis=1)
        else:
            next_step = model.predict([X_start, start_token])
            pred = np.append(pred, next_step, axis=1)

```

```

X_start = np.concatenate((X_start[:,1:], next_step), axis=1)
start_token = X_start[:, -1:]

# Napovedi reskaliramo v osnovno velikost
predictions = []

for i in range(N):
    predictions.append(scaler.inverse_transform(pred[i,:,:]))
T.append((t+1)*0.01)

return np.array(T), np.array(predictions)

```

Tako kot med treningom v nevronske mreže naenkrat pošljemo celoten `batch`, tukaj v model hkrati vstopijo vsi začetni pogoji, s čimer zelo pospešimo testiranje nevronske mreže. V prvi iteraciji v mrežo vstopi zaporedje m “resničnih” stanj iz klasičnega integratorja, v drugi zadnjih $m - 1$ resničnih stanj in prvo napovedano stanje iz RNN, itd. Lahko bi tudi poenostavili postopek, tako da bi si po prvi napovedi shranili skrito stanje RNN in tega nato nastavili kot skrito stanje pred naslednjo napovedjo, vendar je implementacija te poenostavitve v Tensorflowu precej zahtevnejša in presega obseg te naloge. Pri implementaciji bodi pozoren na komentarje in razlike med RNN in modelom z Luongovo pozornostjo (v funkciji napoved tudi ustrezno izberi `Luong_attention=True` ali `False`).

4 Naloga

Cilj naloge je ustvariti in raziskovati nevronske integratorje Lorenzovega sistema enačb 9. Najprej z integriranjem sistema z natančnim klasičnim integratorjem (npr. Runge-Kutta 5. reda `dopri5` v `odeint`) in kratkim časovnim korakom $\Delta t = 10^{-2}$ ustvari dovolj dolgo učno množico (Dodatek B). Uporabi vrednosti parametrov $\sigma = 10$, $b = 8/3$ in $r = 28$.

Nato raziskuj nevronske mreže skladno s spodnjimi nalogami. O kakovosti njihovih napovedi ti veliko pove že njihov “`val_mean_squared_error`”. Ta opiše kakovost napovedi prvega časovnega koraka na validacijski množici. Vrednosti funkcije izgube izriši na učni in validacijski množici. Za boljši pregled napovedne zmogljivosti modela z nevronske mreže na testni množici izvedi integracijo v času in za čase $t = \Delta t, 2\Delta t, \dots, 10$ riši njihovo rast napake:

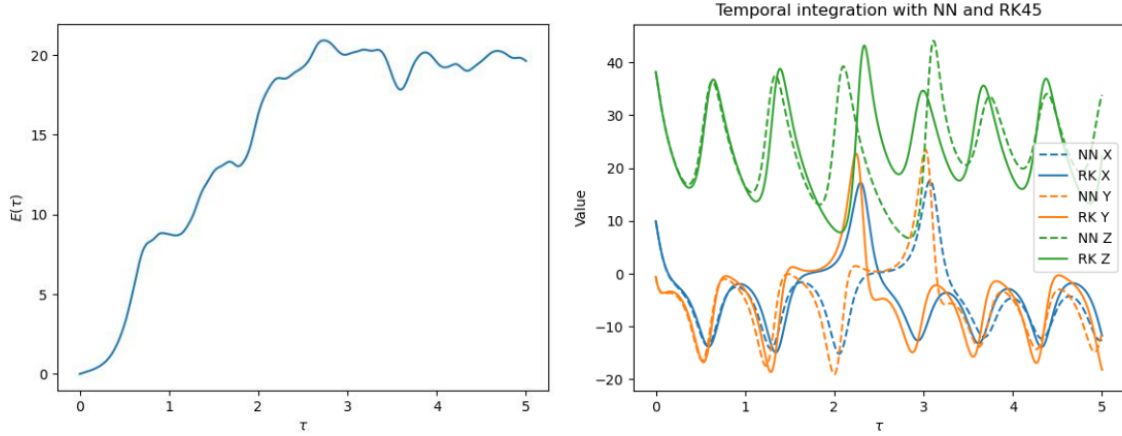
$$E(t) = \sqrt{\frac{1}{N} \sum_{i=1}^N \left((X_i^f(t) - X_i^t(t))^2 + (Y_i^f(t) - Y_i^t(t))^2 + (Z_i^f(t) - Z_i^t(t))^2 \right)}, \quad (20)$$

kjer z nadnapisom f označimo napoved iz nevronske mreže (*forecast*), s t označimo “resnico” (*truth*) iz klasičnega numeričnega integratorja, N pa je število napovedi za izračun napake (št. začetnih pogojev iz testne množice, vzemi npr. $N = 200$). Za katero od napovedi nariši tudi kvalitativni razvoj stanja skozi čas, kot npr. na sliki 6.

Najprej reši obvezne naloge, nato pa podrobneje preuči katero izmed izbirnih nalog.

Obvezne naloge:

1. Primerjaj napoved `SimpleRNN` s klasičnim integratorjem, izriši odvisnost napake (izračunane z metriko 20) od časa.
2. Implementiraj Luongovo pozornost za LSTM in preveri ali pozornost izboljša napoved običajne LSTM.



Slika 6: Desno: Primer integracije sistema v času z nevronske mrežo (NN) in klasičnim integratorjem (RK) za vse 3 komponente. Levo: Primer izračunane napake napovedi s to nevronske mrežo prek metrike 20.

3. Izberi model, ki se ti zdi najbolj perspektiven, in izvedi optimizacijo hiperparametrov:

- velikost skritega stanja,
- `batch_size`,
- načrtovanje stopnje učenja (`callbacks`),
- dolžina vhodne sekvence (`input_seq_len`).

Izbirne naloge: (več * pomeni večjo okvirno zahtevnost naloge)

1. (*) Nevronske mreže uči na parih, ki so več časovnih korakov ($n\Delta t$) narazen:

$$\mathbf{X}(t) \xrightarrow{NN} \mathbf{X}(t + n\Delta t). \quad (21)$$

Pri izračunu napovedi torej korakaš z daljšimi časovnimi koraki. Lahko poskusiš z $n = 10$ in $n = 100$. Kaj se zgodi z $E(t)$?

2. (**) Kaj se zgodi, če problem namesto v obliki $\mathbf{X}(t + \Delta t) = \mathcal{NN}(\mathbf{X}(t))$ formuliramo kot $\mathbf{X}(t + \Delta t) = \mathbf{X}(t) + \mathcal{NN}(\mathbf{X}(t))$. Tedaj torej napovedujemo residue oz. difference $\Delta\mathbf{X}(t) = \mathbf{X}(t + \Delta t) - \mathbf{X}(t)$:

$$\begin{bmatrix} \mathbf{X}(t) \\ \mathbf{X}(t - \Delta t) \\ \mathbf{X}(t - 2\Delta t) \\ \vdots \\ \mathbf{X}(t - (k-1)\Delta t) \end{bmatrix} \xrightarrow{\mathcal{NN}} \Delta\mathbf{X}(t). \quad (22)$$

Kdaj oz. zakaj bi ta pristop lahko bil boljši? (namig pri učenju: pazi na normalizacijo izhodov!).

3. (*) Napovedi naučenega modela se nahajajo blizu minimuma napake napovedi na učni in validacijski množici, vendar kljub temu njegove uteži niso natančno optimalne, še posebej na testni množici. Ali lahko namesto deterministične napovedi z optimalnim modelom, napoved ustvarimo z množico modelov (ansamblom), katerih uteži rahlo perturbiramo.

Določi smiselno standardno deviacijo perturbacij uteži in primerjaj povprečno napoved ansambla z napovedjo “optimalno” naučenega modela. Postopek perturbacije uteži je zapisan v dodatku C.

4. (*) Koliko pozornosti model namenja različnim skritim stanjem kodirnika? Izračun vrednosti uteži pozornosti dobimo med izvajanjem napovedi, tako da pred časovno for zanko definiramo kodirnik, dekodirnik in sloj pozornosti, znotraj časovne zanke pa izračunamo uteži - glej dodatek D. Ali se zdijo vrednosti smiselne?

Dodatek A

Približek Navier-Stokesove enačbe za majhne perturbacije

Pritisk in gostoto lahko razpišemo kot del, ki se spreminja zgolj z višino in deviacijo od te vrednosti:

$$\begin{aligned} p(x, y, z, t) &= p_0(z) + p'(x, y, z, t) \\ \rho(x, y, z, t) &= \rho_0(z) + \rho'(x, y, z, t). \end{aligned} \quad (23)$$

V smeri $\hat{k}$ del desne strani enačbe (1) razpišemo kot

$$\begin{aligned} -\frac{1}{\rho} \frac{\partial p}{\partial z} - g &= -\frac{1}{\rho_0 + \rho'} \frac{\partial}{\partial z} (p_0 + p') - g \\ &\approx \frac{1}{\rho_0} \left(\frac{\rho'}{\rho_0} \frac{\partial p_0}{\partial z} - \frac{\partial p'}{\partial z} \right) = -\frac{1}{\rho_0} \left(\rho' g + \frac{\partial p'}{\partial z} \right), \end{aligned} \quad (24)$$

kjer smo upoštevali $(1/\rho_0)(\partial p_0/\partial z) = -g$ ter $(\rho_0 + \rho')^{-1} \approx \rho_0^{-1}(1 - \rho'/\rho_0)$ in zanemarili člene drugega reda. Variacije gostote ρ' pa pripišemo variacijam temperature, torej

$$\rho' = \rho - \rho_0 = -\beta \rho_0 (T - T_0). \quad (25)$$

Velja torej

$$-\frac{1}{\rho} \frac{\partial p}{\partial z} - g = -\frac{1}{\rho_0} \frac{\partial p'}{\partial z} + g\beta(T - T_0). \quad (26)$$

Dodatek B

Integracija Lorenzovega sistema z dopri5

```
from scipy.integrate import ode
import numpy as np

def integratorLorenz63(
    X0, Y0, Z0, integration_time,
    sigma=10., b = 8./3, r=28.
):
    """
    Integracija Lorenzovega '63 modela
    Args:
    - X0, Y0, Z0 = X(t=0), Y(t=0), Z(t=0) -> začetni pogoji
    - integration_time -> 1 enota = 100 korakov pri dt=0.01
    - sigma, b, r -> konstante sistema enačb
    """
```



```

# Funkcija za integracijo: parcialno X/parcialno tau
def f(t, X):
    # Desna stran Lorenz '63 sistema
    return [sigma*(X[1]-X[0]),
            X[0]*(r-X[2])-X[1],
            X[0]*X[1]-b*X[2]]

def jac(t, X):
    # Jakobian sistema
    return [[-sigma, sigma, 0],
            [r-X[2], X*(r-X[2])-1, -X[0]],
            [X[1], X[0], -b]]

# Integrator Runge-Kutta reda (4)5
Integrator = ode(f, jac).set_integrator('dopri5')

# Začetni pogoji za integracijo
X0 = [X0, Y0, Z0] # Pozor - tu redefiniramo X0
t0 = 0
Integrator.set_initial_value(X0, t0)

# Časovni korak integracije
dt = 0.01

# Tabeli za shranjevanje rezultatov integracije
t = [0.]
X = [X0]

# Integracija do časa t + dt
while Integrator.successful() and Integrator.t+dt < integration_time:
    t.append(float(Integrator.t+dt))
    X.append(list(Integrator.integrate(Integrator.t+dt)))

return np.array(t), np.array(X)

# Za ponovljivost določimo seme
np.random.seed(42)

#Začetni pogoji za učno trajektorijo
X0_train = float(np.random.uniform(low=-50., high = 50., size = 1))
Y0_train = float(np.random.uniform(low=-50., high = 50., size = 1))
Z0_train = float(np.random.uniform(low=-50., high = 50., size = 1))

# Čas integracije
t_train = 8030. # Prvih 30 enot časa bomo odstranili

# Numerična integracija
t_train, data_train = integratorLorenz63(

```

```

X0_train, Y0_train, Z0_train,
t_train)

# Odrežemo prvih 30 enot časa - da ohranimo le točke na atraktorju
relaksacijski_indeks = 3000
data_train = np.array(data_train)[relaksacijski_indeks:, :]

# Podatke razdelimo na učno in validacijsko množico
# 90 % za učenje, 10 % za validacijo
split_idx = int(0.1 * len(data_train))

X_val = data_train[:split_idx]
X_train = data_train[split_idx:]

```

Če generiranje podatkov traja daljši čas jih lahko tudi shranimo.

Dodatek C

Perturbacija uteži

Modelu perturbiramo uteži na sledeči način:

```

# Loadaj model
model_perturbed = tf.keras.models.load_model("pot_naučenega_modela")
# Dostopaj do uteži
weights = model_perturbed.get_weights()

# Standardna deviacija perturbacij
perturbation_std = 0.01

# Perturbiraj uteži z Gaussovskim šumom
perturbed_weights = []
for weight in weights:
    noise = np.random.normal(
        loc=0.0, scale=perturbation_std, size=weight.shape)
    perturbed_weights.append(weight + noise)

# Vstavi perturbirane uteži v model
model_perturbed.set_weights(perturbed_weights)

```

Dodatek D

Vrednosti uteži pozornosti

Uteži pozornosti dobimo z uporabo naslednjih funkcij:

```

def komponente(model):
    """
    Funkcija, ki vrne kodirnik, dekodirnik in instanco za izračun pozornosti.
    """

```

```

# Z naučenim slojem "encoder" ustvarimo kodirnik
encoder_model = tf.keras.Model(
    inputs=model.input[0],
    outputs=model.get_layer('encoder').output)# [0])
# model.input[0] je encoder_inputs iz definicije modela:
# tf.keras.Model(inputs=[encoder_inputs, decoder_inputs],
# outputs = outputs))
# model.get_layer('encoder').output je celoten output kodirnika,
# torej encoder_hidden_states, state_h, state_c

# Z naučenim slojem "decoder" ustvarimo dekodirnik
decoder_model = tf.keras.Model(
    inputs=[model.input[1], model.get_layer('encoder').output[1:3]],
    outputs=model.get_layer('decoder').output[0])
# model.input[1] = decoder_inputs
# model.get_layer('decoder').output je celoten output dekodirnika,
# torej decoder_hidden_states, state_h, state_c

# Instanca za izračun pozornosti
attention_layer = LuongAttention()

return encoder_model, decoder_model, attention_layer

def utezi_pozornosti(
    encoder_model, decoder_model, attention_layer, start_token, X_start):
    """Funkcija, ki vrne vrednosti uteži pozornosti"""
    # Izračunaj stanja kodirnika
    encoder_hidden_states, state_h, state_c = encoder_model.predict(X_start)
    # Izračunaj skrita stanja dekodirnika
    decoder_hidden_states = decoder_model.predict(
        [start_token, state_h, state_c])
    # Izračunaj vektor konteksta in uteži pozornosti
    context_vector, attention_weights = attention_layer(
        decoder_hidden_states, encoder_hidden_states)

    return attention_weights

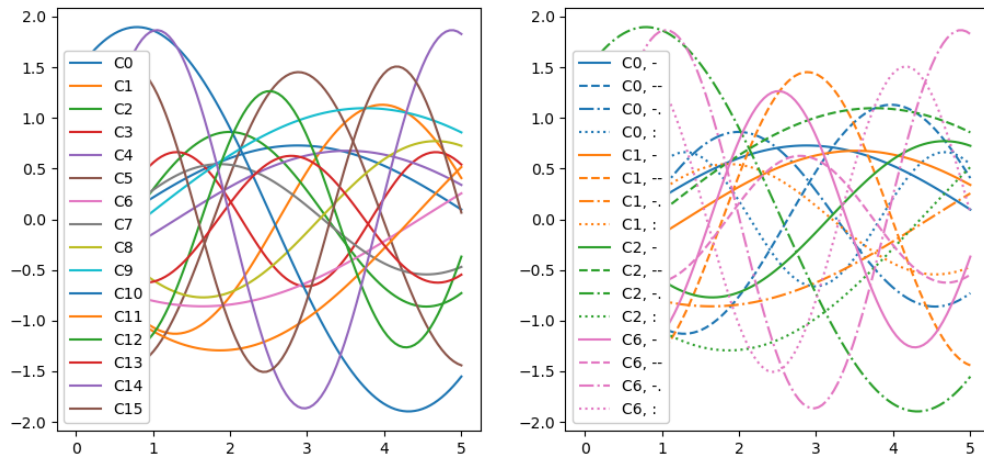
```

Pri tem potrebujemo naučeni model z Luongovo pozornostjo, `start_token` in `X_start`. Funkciji je smiselno klicati znotraj funkcije "napoved", na mestu, kjer so izračunani vsi potrebni argumenti.

Dodatek E

Barvno slepim prijazni grafi

Približno 8 % moških in 0.5 % žensk naj bi imelo eno izmed oblik barvne slepote. Mnoge strokovne revije zato od avtorjev pričakujejo barvno slepim prijazne grafe (na kratko: čim manj naj se prepletata rdeča in zelena). Demonstracija, kako dobiti graf z mnogimi črtami, ki jih razloči večina barvno slepih, je na sliki 7.



Slika 7: Levo: prikaz 16 črt s standardnim zaporedjem barv v Matplotlib. Desno: enake črte, le predstavljene na barvno slepim prijazen način. Legenda na desni vsebuje (color, linestyle).

Literatura

- [1] Edward N Lorenz. Deterministic nonperiodic flow. *Journal of atmospheric sciences*, 20(2): 130–141, 1963.
- [2] N. Žagar, A. Kasahara, K. Terasaki, J. Tribbia, and H. Tanaka. Normal-mode function representation of global 3-D data sets: Open-access software for the atmospheric research community. *Geoscientific Model Development*, 8(4):1169–1195, 2015. ISSN 19919603. doi: 10.5194/gmd-8-1169-2015.
- [3] Yaşar Demirel and Vincent Gerbaud. *Nonequilibrium Thermodynamics (Fourth Edition)*. Elsevier, fourth edition edition, 2019. ISBN 978-0-444-64112-0. doi: <https://doi.org/10.1016/B978-0-444-64112-0.00013-7>. URL <https://www.sciencedirect.com/science/article/pii/B9780444641120000137>.
- [4] Rishi G. Image caption generation methodologies. 04 2021.
- [5] Minh-Thang Luong, Hieu Pham, and Christopher Manning. Effective approaches to attention-based neural machine translation. 08 2015. doi: 10.18653/v1/D15-1166.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.